Rumos



Java Backend com Spring Boot

Novembro de 2026

Siga-nos —

C2 General

Agenda da Formação

01. Spring Boot & Rest Básico

02. API Design, DTOs & Validação

03. Persistência com JPA & Hibernate

04. Integração Externa (TMDB)

05. Qualidade, Erros & Testes

06. Spring Security, JWT & OAuth2

07. Empacotamento & Container

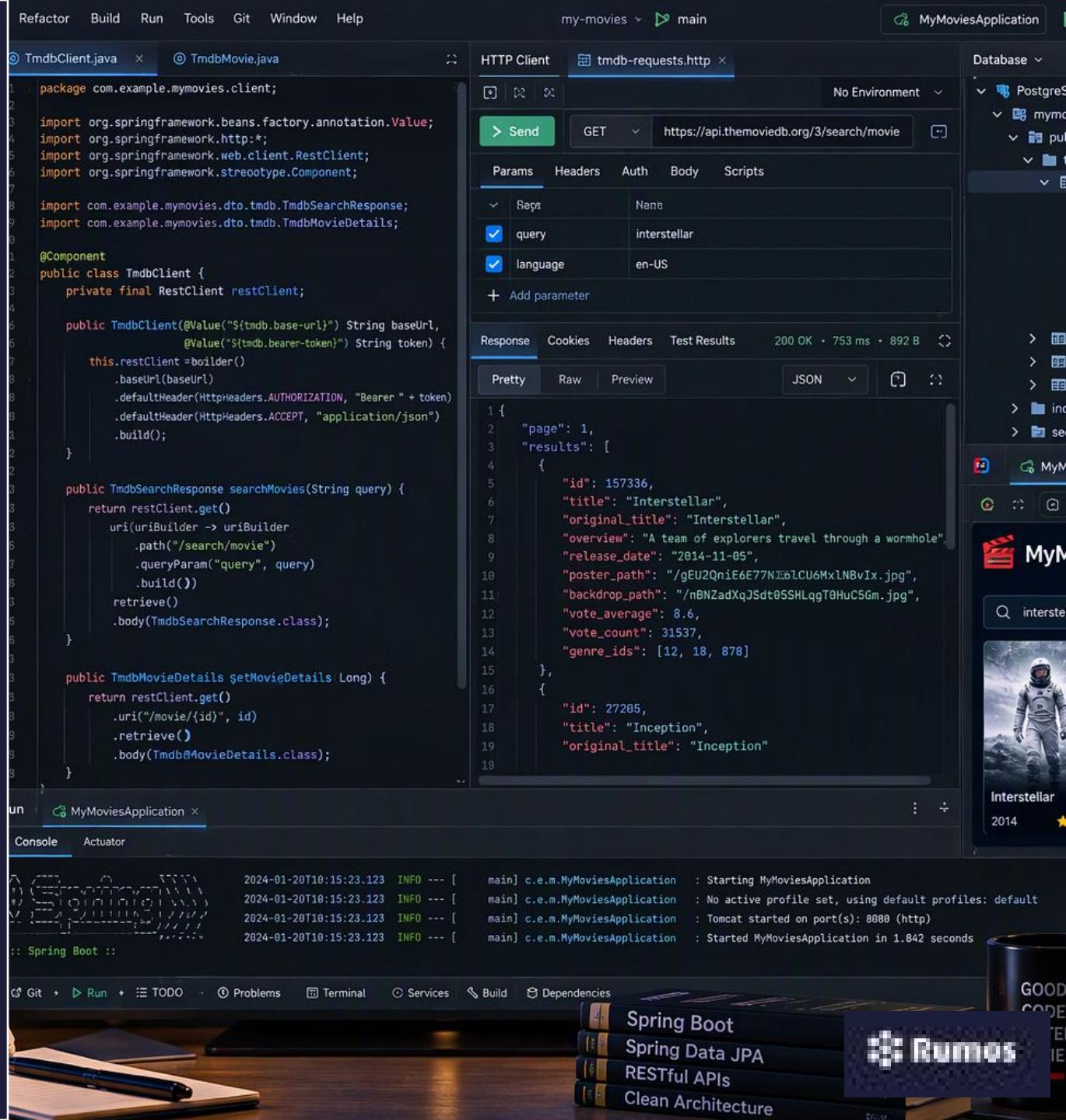
08. Kubernetes Local (Minikube)

Rumos

ACADEMIA JAVA

Integração externa com TMDB

Consumir dados reais sem entregar o nosso domínio ao fornecedor.



Preparação – Cadastro, Gerar Tokens, Teste

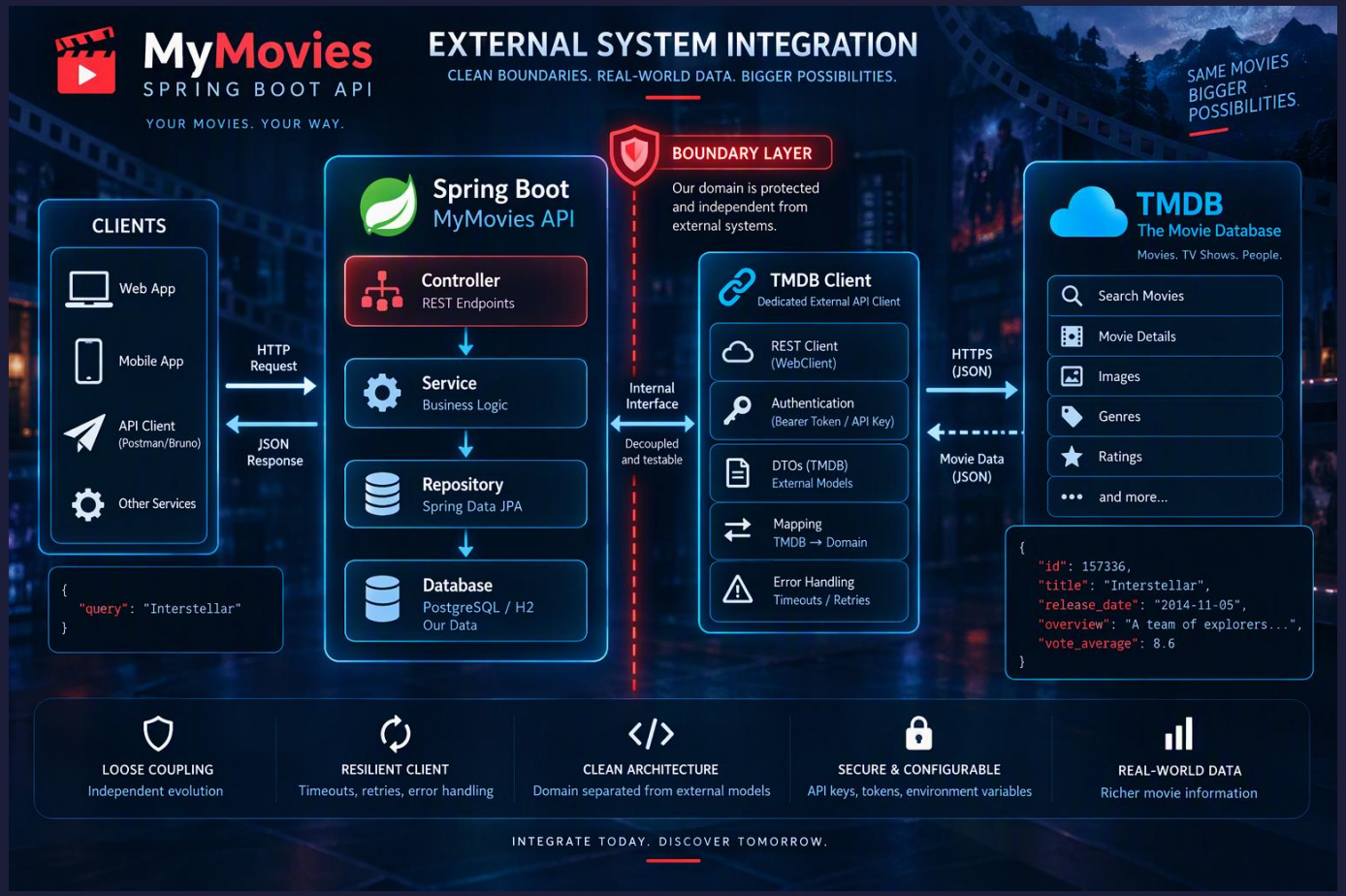
```
Conta TMDB confirmada
Credenciais próprias disponíveis
Testes com Bruno
```

API Read Access Token
API Key v3
Cada aluno usa a sua conta.
Ainda sem executar código Java.

A dinâmica chama o TMDB diretamente.

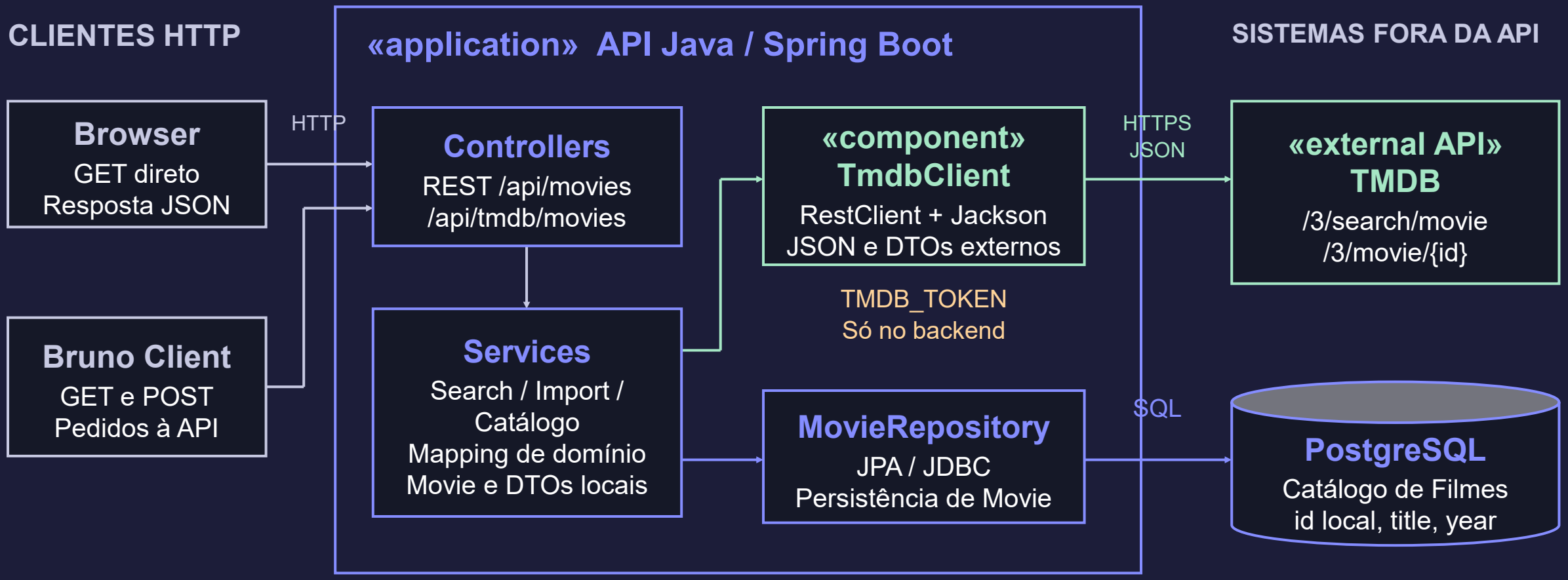
Nunca partilhar tokens, chaves ou capturas de ecrã que revelem os segredos.

A dependência externa deve ficar atrás de uma boundary



Rumos

MyMovies: componentes e integração externa



Objetivo: Search consulta o TMDB sem persistir. Import obtém details, converte para Movie e guarda no SGBD.

O Spring cria e liga os objetos por nós

1

Component scan

O Spring procura classes anotadas (@Component e derivadas) a partir do pacote da aplicação.

2

Criação do bean

O container instancia a classe uma vez (singleton) e guarda-a no ApplicationContext.

3

Injeção de dependências

O que o construtor pede é entregue pelo container. Sem usar new.

Bean = objeto criado, configurado e gerido pelo container IoC (ApplicationContext).
Component = classe que o Spring regista automaticamente como bean.

@Component e as suas especializações

@Component
Bean genérico (ex.: TmdbClient)

@Controller
Camada web: recebe pedidos HTTP
Ex.: /api/movies

@Service
Regras de negócio e orquestração
Ex.: Search, Import

@Repository
Acesso a dados; traduz exceções de persistência
Ex.: MovieRepository

@Configuration
Declara beans com métodos @Bean
Ex.: beans de bibliotecas externas

@RestController
= @Controller + @ResponseBody

Todos são beans: o Spring deteta-os via @ComponentScan, já incluído em @SpringBootApplication.

A anotação escolhida documenta a camada do código.

Roteiro: 8 peças de código, por ordem

FASE 1 / SEARCH: só lê do TMDB, não guarda nada

1 TmdbClient
Configura URL e Bearer

2 DTOs externos
Records mínimos com Jackson

3 search()
Chama /search/movie

4 Service + GET
Devolve MovieCandidate

FASE 2 / IMPORT: cria o filme no nosso catálogo

5 details(id)
Lê o filme escolhido

6 Mapping
release_date → year

7 ImportService
Cria Movie e guarda

8 POST /import
Devolve 201 + MovieResponse

Cada peça só conhece a vizinha: Controller → Service → TmdbClient → TMDB.

Fluxo TMDB: primeiro search, depois details

1 `GET /search/movie?query=…` → **results[] + id**

2 `GET /movie/{id}` → **movie details**

A própria documentação TMDB apresenta este padrão.

https://developer.themoviedb.org/docs/search-and-query-for-details

Credencial: configuração, não código



Configuração segura da credencial

application.properties

```
tmdb.base-url=https://api.themoviedb.org/3
tmdb.token=${TMDB_TOKEN:}
```

IntelliJ
Run Configuration
Environment variables
TMDB_TOKEN

O valor real fica no ambiente.
O ficheiro versionado contém apenas a referência à variável.

Nunca projetar tokens, headers completos ou configurações com segredos.

Verificação antes da primeira chamada Java

Verificação	Evidência esperada
Credencial no ambiente	A aplicação recebe TMDB_TOKEN sem o expor.
URL base	Host oficial TMDB e prefixo /3.
Header	Authorization usa Bearer e o token configurado.

Onde poderia uma credencial entrar acidentalmente no Git?

TmdbClient concentra a configuração HTTP

```
@Component
public class TmdbClient {
  private final RestClient client;

  public TmdbClient(RestClient.Builder builder,
      @Value("${tmdb.base-url}") String url,
      @Value("${tmdb.token}") String token) {

      client = builder.baseUrl(url)
        .defaultHeader(HttpHeaders.AUTHORIZATION,
            "Bearer " + token).build();
  }
}
```

Responsabilidade

Conhecer HTTP, URL e autenticação.

Dependência

O Service recebe TmdbClient.

O Controller recebe o Service.

```
//@Bean para o Spring saber como construir o RestClient.Builder

@Bean
public RestClient.Builder restClientBuilder() {
    return RestClient.builder();
}
```

Não modele o nosso sistema com o JSON do fornecedor

TMDB pode devolver muitos campos.

- id
- title
- original_title
- overview
- release_date
- poster_path
- genre_ids
- vote_average
- …

Precisamos apenas do necessário para a funcionalidade atual.

External DTO → mapping → nosso modelo

Jackson: JSON externo e records mínimos

Fixture fictícia e sanitizada

```
{
  "results": [{
    "id": 101,
    "title": "Filme de exemplo",
    "release_date": "2020-06-12",
    "overview": "Campo ignorado"
  }]
}
```

Jackson 2: anotações nos DTOs externos

```
@JsonIgnoreProperties(ignoreUnknown = true)
public record TmdbSearchResponse(
    List<TmdbMovie> results) {}

-------------------------------------

@JsonIgnoreProperties(ignoreUnknown = true)
public record TmdbMovie(
    long id, String title,
    @JsonProperty("release_date")
    String releaseDate) {}
```

O id 101 pertence apenas à fixture. Na demo, escolher um id real da pesquisa.

As anotações definem a leitura do JSON

```
@JsonIgnoreProperties(ignoreUnknown = true)
public record TmdbMovie(
    long id,
    String title,
    @JsonProperty("release_date")
    String releaseDate) {}
```

@JsonProperty
Associa release_date a releaseDate.

ignoreUnknown
Ignora campos extra.

releaseDate continua String. A conversão para year pertence ao domínio.

O método search recebe uma intenção de pesquisa

```
public TmdbSearchResponse search(String query) {
  return client.get()
      .uri(uri -> uri.path("/search/movie")
          .queryParam("query", query)
          .build())
      .retrieve()
      .body(TmdbSearchResponse.class);
}
```

Entrada

Texto da pesquisa.

Saída

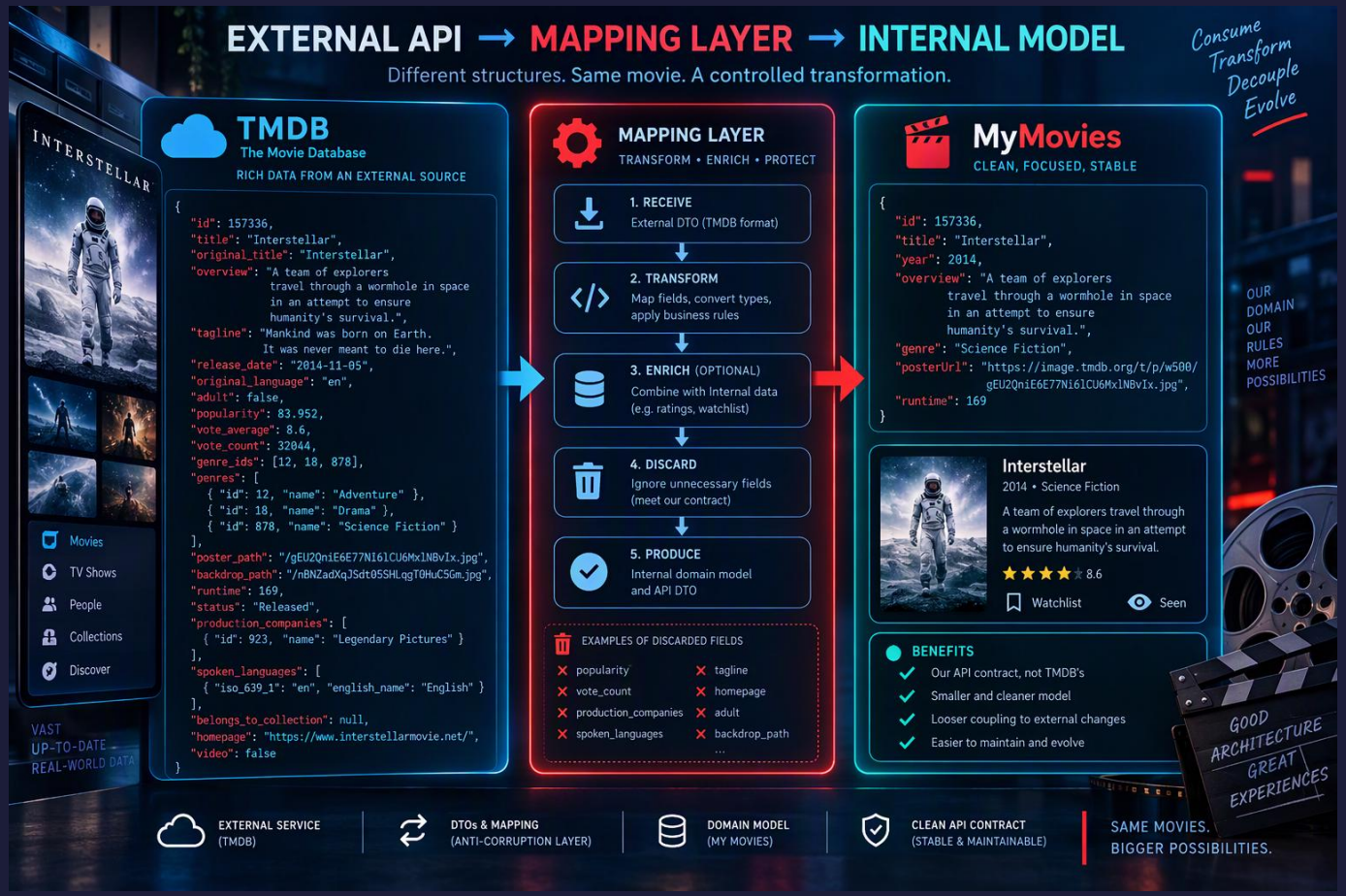
Resposta externa tipada.

URI builder

Transporta query como parâmetro.

body(TmdbSearchResponse.class) usa o conversor JSON configurado com Jackson.

DTO externo é tradução, não contrato público



SearchService traduz o DTO externo em contrato local

```
// SearchService
public List<MovieCandidate> search(String query) {

   return tmdbClient.search(query)
      .results()
      .stream()
      .map(m -> new MovieCandidate(m.id(), m.title()))
      .toList();
}
```

Entrada

DTO externo do TMDB.

Saída

MovieCandidate local.

id externo vira tmdbId.

A pesquisa entra pelo endpoint local

```
@GetMapping("/api/tmdb/movies/search")
public List<MovieCandidate> search(
    @RequestParam String query) {
  return searchService.search(query);
}

public record MovieCandidate(
    long tmdbId, String title) {}
```

Controller
Recebe query e delega.

SearchService
Chama TmdbClient e traduz os resultados.

MovieCandidate
Contrato público local.

Exemplo de contrato do curso. O Service traduz id externo para tmdbId.

CONCEPT

Dois endpoints, duas responsabilidades

GET	`/api/tmdb/movies/search?query=…`	descobrir candidatos externos
POST	`/api/tmdb/movies/{tmdbId}/import`	criar recurso local a partir do candidato

Search não persiste. Import cria algo que passa a ser nosso.

O id selecionado alimenta a chamada details

```
public TmdbMovie details(long tmdbId) {
  return client.get()
      .uri("/movie/{id}", tmdbId)
      .retrieve()
      .body(TmdbMovie.class);
}
```

Entrada
Id de um candidato.

Resposta
Detalhes externos do filme.

Ainda sem persistência.

CODE

Mapping de domínio: DTO externo para Movie

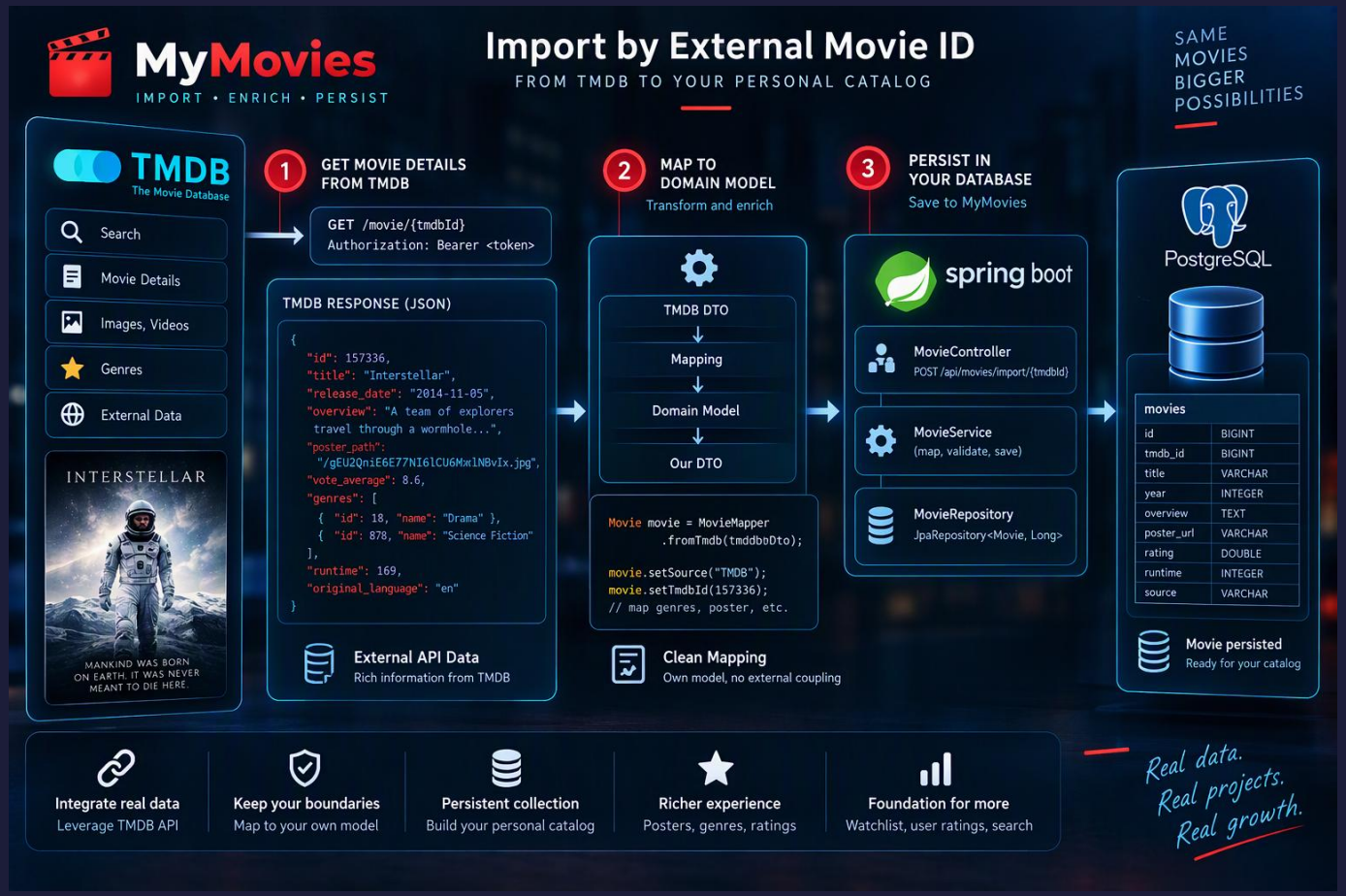
```
1 TmdbMovie external = tmdbClient.details(tmdbId);
2
3 int year = LocalDate
4                 .parse(external.releaseDate())
5                 .getYear();
6
7 Movie movie = new Movie(external.title(), year);
```

- Jackson entrega o DTO externo.
- O mapper de domínio extrai year.
- O id externo serve ao lookup.
- Data nula ou inválida exige decisão.

Campos e conversões

Entrada externa	Destino local	Decisão
title	Movie.title	Copiar o título.
release_date	Movie.year	Extrair o ano de uma data válida.
Id	Seleção de import	Usar para details, sem substituir o id local.
vote_average	Movie.rating	Extrai a nota do filme
overview e restantes	Sem destino neste caso	Ignorar se não forem necessários.

Import: lookup externo → mapping → persistência local



O Service coordena mapping e persistência

```
public MovieResponse importMovie(long tmdbId) {
  TmdbMovie external = tmdbClient.details(tmdbId);
  int year = LocalDate.parse(external.releaseDate()).getYear();

  Movie movie = new Movie(external.title(), year);

  //Cria Dto movieResponse a partir de Movie

  return movieRepository.save(movieResponse);
}
```

TmdbClient
Obtém detalhes.

Service
Converte e cria Movie.
Retorna Dto

Repository
Guarda no catálogo local.

O endpoint de import devolve o contrato local

```
@PostMapping("/api/tmdb/movies/{tmdbId}/import")
public ResponseEntity<MovieResponse> importMovie(
                          @PathVariable long tmdbId) {

  MovieResponse saved = tmdbService.importMovie(tmdbId);

  return ResponseEntity.status(HttpStatus.CREATED).body(body);
}
```

201 Created
Depois da persistência local.

MovieResponse
DTO público da aplicação.

Lab: search primeiro, import depois

Core

- Criar TmdbClient.
- Configurar Authorization Bearer via TMDB_TOKEN.
- Implementar search(query).
- Criar GET /api/tmdb/movies/search.
- Criar POST /api/tmdb/movies/{tmdb_Id}/import
- Verificar sucesso de importacao local

Prova de que o filme pertence ao catálogo local

1. Escolher um candidato no resultado do search

2. Executar o endpoint de importação

```
POST /api/tmdb/movies/{tmdbId}/import
```

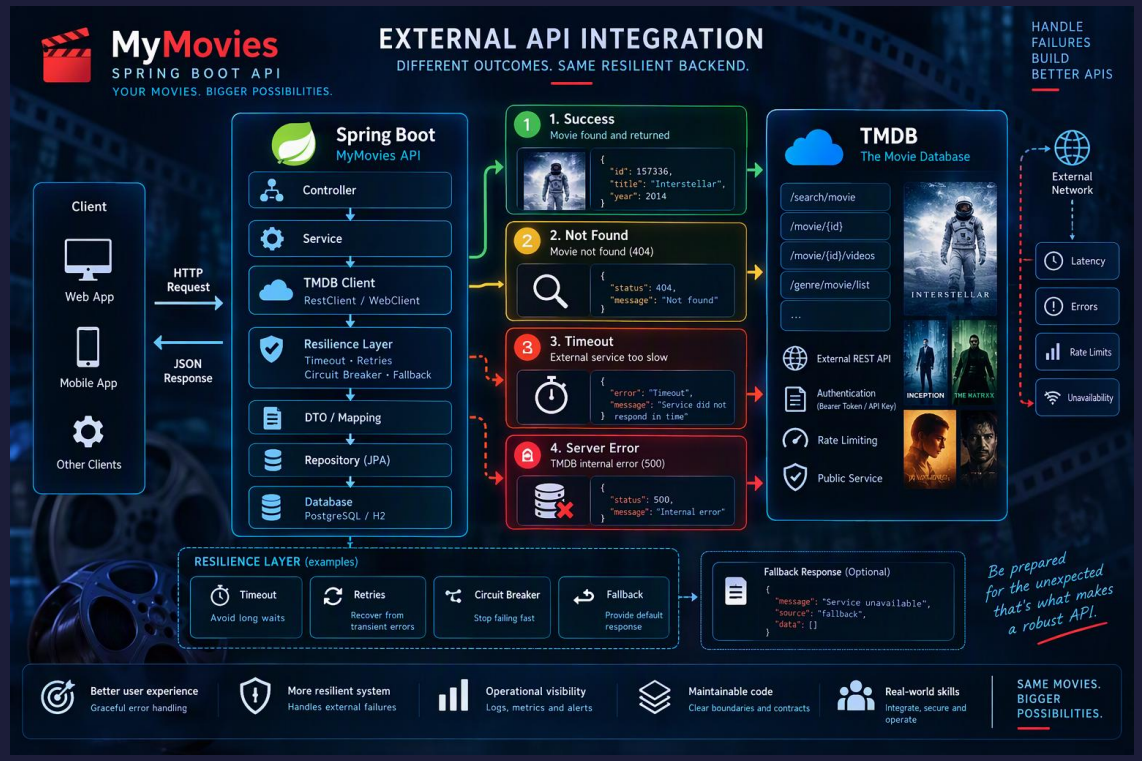
3. Consultar o catálogo depois do import

```
GET /api/movies
```

Evidência: título, ano, nota e identidade local aparecem na resposta do catálogo.

Dependência externa significa novos modos de falha

Credencial ausente ou inválida.
404 no lookup externo.
Timeout ou indisponibilidade.
Mudança de schema.
Resposta vazia.
Data nula ou inesperada.



Hoje: reconhecer as falhas. Aula 5: definir o tratamento consistente.

Rumos

Timeout é parte do contrato operacional

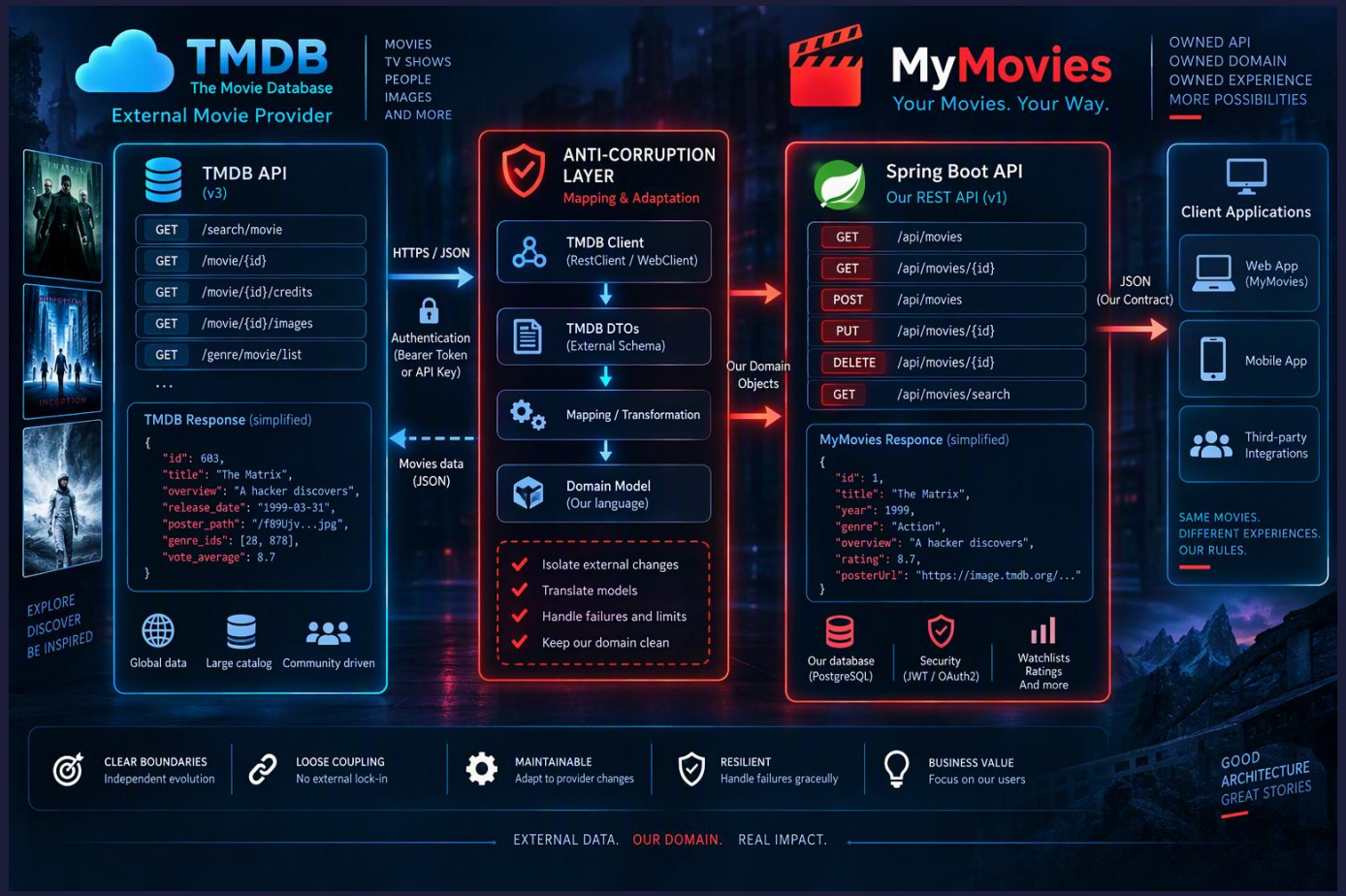
Sem limite, quanto tempo aceitamos manter uma request à espera do fornecedor?

Connect timeout

Read timeout

Checkpoint: configuração mínima.
Produção: escolha explícita, teste e observabilidade.

Architect View: o fornecedor não é dono do nosso modelo



IA Tutor: reveja o acoplamento, não gere o client

“Compare este DTO TMDB com o nosso Movie. Identifique acoplamentos e proponha um mapping mínimo. Não gere o cliente HTTP.”

- Quais campos realmente pertencem ao nosso caso de uso?
- Quais formatos exigem conversão?
- Onde podem existir nulls ou mudanças externas?
- Que detalhe do fornecedor está vazando?

Checkpoint: aula-04-tmdb

- ✓ TMDB_TOKEN fora do source
- ✓ TmdbClient dedicado
- ✓ search real via RestClient
- ✓ DTO externo separado
- ✓ details + mapping
- ✓ import persiste recurso local

`checkpoint/aula-04-tmdb`

Próxima aula: quando algo falha, a API precisa responder de forma previsível.